PAYSIM FRAUD ANALYSIS

BY MARY KANE

Importing libraries and Dataset

In [1]:
```
import pandas as pd
import numpy as np
```

In [2]:
```
df = pd.read_csv("paysim[1].csv")
```

In [3]:
```
df.head()
```

Out[3]:

	step	type	amount	nameOrig	oldbalanceOrg	newbalanceOrig	nameDest
0	1	PAYMENT	9839.64	C1231006815	170136.0	160296.36	M1979787155
1	1	PAYMENT	1864.28	C1666544295	21249.0	19384.72	M2044282225
2	1	TRANSFER	181.00	C1305486145	181.0	0.00	C553264065
3	1	CASH_OUT	181.00	C840083671	181.0	0.00	C38997010
4	1	PAYMENT	11668.14	C2048537720	41554.0	29885.86	M1230701703

Understanding the Data

In [5]:
```
# Data Size
df.shape
```

Out[5]:
```
(6362620, 11)
```

In [6]:
```
# Column Names
df.columns
```

Out[6]:
```
Index(['step', 'type', 'amount', 'nameOrig', 'oldbalanceOrg', 'newbalanceOrig',
       'nameDest', 'oldbalanceDest', 'newbalanceDest', 'isFraud',
       'isFlaggedFraud'],
      dtype='object')
```

In [8]:
```
# Data types
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6362620 entries, 0 to 6362619
Data columns (total 11 columns):
 #   Column          Dtype
---  ------          -----
 0   step            int64
 1   type            object
 2   amount          float64
 3   nameOrig        object
 4   oldbalanceOrg   float64
 5   newbalanceOrig  float64
 6   nameDest        object
 7   oldbalanceDest  float64
 8   newbalanceDest  float64
 9   isFraud         int64
 10  isFlaggedFraud  int64
dtypes: float64(5), int64(3), object(3)
memory usage: 534.0+ MB
```

In [10]:
```
# Missing Values
df.isnull().sum()
```

Out[10]:
```
step              0
type              0
amount            0
nameOrig          0
oldbalanceOrg     0
newbalanceOrig    0
nameDest          0
oldbalanceDest    0
newbalanceDest    0
isFraud           0
isFlaggedFraud    0
dtype: int64
```

In [11]:
```
# Duplicate rows
df.duplicated().sum()
```

Out[11]:
```
np.int64(0)
```

Understanding Fraud Distribution

In [12]:
```
# Transactions Types
df["type"].value_counts()
```

Out[12]:
```
type
CASH_OUT    2237500
PAYMENT     2151495
CASH_IN     1399284
TRANSFER     532909
DEBIT         41432
Name: count, dtype: int64
```

In [13]:
```
# Fraud Counts
df["isFraud"].value_counts()
```

Out[13]:
```
isFraud
0    6354407
1       8213
Name: count, dtype: int64
```

In [14]:
```
# Fraud Percentage
df["isFraud"].value_counts(normalize=True) * 100
```

Out[14]:
```
isFraud
0    99.870918
1     0.129082
Name: proportion, dtype: float64
```

Basic Exploration

In [15]:
```
# Summary STATS
df.describe()
```

Out[15]:

	step	amount	oldbalanceOrg	newbalanceOrig	oldbalanceDest	newb
count	6.362620e+06	6.362620e+06	6.362620e+06	6.362620e+06	6.362620e+06	6.3
mean	2.433972e+02	1.798619e+05	8.338831e+05	8.551137e+05	1.100702e+06	1.2
std	1.423320e+02	6.038582e+05	2.888243e+06	2.924049e+06	3.399180e+06	3.6
min	1.000000e+00	0.000000e+00	0.000000e+00	0.000000e+00	0.000000e+00	0.0
25%	1.560000e+02	1.338957e+04	0.000000e+00	0.000000e+00	0.000000e+00	0.0
50%	2.390000e+02	7.487194e+04	1.420800e+04	0.000000e+00	1.327057e+05	2.1
75%	3.350000e+02	2.087215e+05	1.073152e+05	1.442584e+05	9.430367e+05	1.1
max	7.430000e+02	9.244552e+07	5.958504e+07	4.958504e+07	3.560159e+08	3.5

In [17]:
```
# Largest transactions
df.sort_values("amount", ascending=False). head(10)
```

Out[17]:

	step	type	amount	nameOrig	oldbalanceOrg	newbalanceOrig	n
3686583	276	TRANSFER	92445516.64	C1715283297	0.0	0.0	C4
4060598	300	TRANSFER	73823490.36	C2127282686	0.0	0.0	C7
4146397	303	TRANSFER	71172480.42	C2044643633	0.0	0.0	C
3946920	286	TRANSFER	69886731.30	C1425667947	0.0	0.0	C1
3911956	284	TRANSFER	69337316.27	C1584456031	0.0	0.0	C14
3937152	286	TRANSFER	67500761.29	C811810230	0.0	0.0	C17
4105338	302	TRANSFER	66761272.21	C420748282	0.0	0.0	C10
3892529	284	TRANSFER	64234448.19	C1139847449	0.0	0.0	C
3991638	298	TRANSFER	63847992.58	C300140823	0.0	0.0	C5
4143801	303	TRANSFER	63294839.63	C372535854	0.0	0.0	C18

BROZE Layer Findings

- The dataset contains 6,362,620 simulated financial transactions.
- The average transaction amount is about $179,862, but the maximum transaction is over$92 million, showing strong right-skew and extreme outliers.
- Fraud is rare: the mean of isFraud is 0.00129, meaning only about 0.13% of transactions are fraudulent.
- isFlaggedFraud is even rarer, suggesting that the system flag may miss many suspicious transactions.
- Several of the largest transactions are TRANSFER transactions but are not labeled as fraud.
- Many high-value transactions show oldbalanceOrg and newbalanceOrig as 0, which may require additional validation in the Silver layer.

Important Insight

In [18]:
```
isFraud = 0
isFlaggedFraud = 0
```

The largest transactions are HUGE tansfers, but all top 10 shows are labeled "0"

Are high-value transfers always risky, or do balance patterns matter more than amount alone?

In [19]:
```
df["isFraud"].value_counts()
```

Out[19]:
```
isFraud
0    6354407
1       8213
Name: count, dtype: int64
```

In [20]:
```
df.groupby("type")["isFraud"].agg(["count", "sum", "mean"]).sort_values("mean", asc
```

Out[20]:

type	count	sum	mean
TRANSFER	532909	4097	0.007688
CASH_OUT	2237500	4116	0.001840
CASH_IN	1399284	0	0.000000
DEBIT	41432	0	0.000000
PAYMENT	2151495	0	0.000000

Key Insights

High-value transfers are not always fraud, but transaction type and balance behavior matter. Fraud doesn't appear in CASH IN, DEBIT or PAYMENT TRANSFER has the highest fraud rate

TRANSFER fraud rate = 0.77%

CASH_OUT fraud rate = 0.18%

TRANSFER fraud cases = 4,097

CASH_OUT fraud cases = 4,116

BRONZE INSIGHT:

Fraud is highly concentrated in TRANSFER and CASH_OUT transactions. Although high transaction amount alone does not always indicate fraud, transaction type appears to be an important risk factor. TRANSFER transactions have the highest fraud rate at approximately 0.77%, while CASH_OUT transactions account for a similar number of fraud cases but a lower fraud rate of approximately 0.18%.

In [ ]:
```

```